



Control de la secuencia de ejecución

Fundamentos de Lenguajes de Programación – IIC2560
Yadran Eterovic S. (yadran@uc.cl)

Expresiones

Comandos

Comandos para control de secuencia

Programación estructurada

Recursión

Subprogramas

Funciones de mayor orden

Excepciones

Las *expresiones* —ya sean numéricas o simbólicas— están presentes en todos los lenguajes

Una **expresión** es una entidad sintáctica cuya evaluación produce un valor

(... o no termina → la expresión queda indefinida)

P.ej., $4+3*2$

P.ej., (cons a b)

Las *expresiones* —ya sean numéricas o simbólicas— están presentes en todos los lenguajes

Una **expresión** es una entidad sintáctica cuya evaluación produce un valor característica distintiva

(... o no termina $\rightarrow$ la expresión queda indefinida)

P.ej.,

$4+3*2 \rightarrow 10$

(cons a b) $\rightarrow$ el par formado por a y b

Un **comando** es una entidad sintáctica cuya evaluación puede tener un *efecto lateral* ... y no necesariamente devuelve un valor

Un **comando** es una entidad sintáctica cuya evaluación puede tener un *efecto lateral* ... y no necesariamente devuelve un valor

Influye en el resultado de la computación, ... pero su evaluación no devuelve ningún valor al contexto en el cual está ubicado:

- p.ej., comandos de output

Una definición precisa y una diferenciación exacta entre expresiones y comandos requieren una definición formal de un lenguaje:

- La evaluación de una expresión, dado un *estado* inicial, produce tanto un valor como un *estado*
- La evaluación de un comando, dado un *estado* inicial, es un nuevo *estado*, producido por los efectos laterales del comando

Una definición precisa y una diferenciación exacta entre expresiones y comandos requieren una definición formal de un lenguaje:

- La evaluación de una expresión, dado un *estado* inicial, produce tanto un valor como un *estado*
- La evaluación de un comando, dado un *estado* inicial, es un nuevo *estado*, producido por los efectos laterales del comando

Estado

Secuencia finita de pares (X, n) : "en el estado actual, la variable X tiene el valor n "

El estado toma en cuenta el valor de todas las variables del programa

El *comando de asignación* es el constructo elemental para modificar el estado

Variable

En matemáticas, un nombre que representa un valor, tomado de algún dominio predeterminado

En lenguajes de programación, sigue siendo un nombre que representa un objeto ... pero además ...

Variable

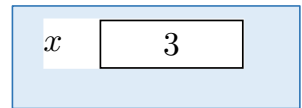
En matemáticas, un nombre que representa un valor, tomado de algún dominio predeterminado

En lenguajes de programación, sigue siendo un nombre que representa un objeto ... pero además ...

Variable modificable

Nombre que representa una parte de almacenamiento modificable, una suerte de contenedor, o ubicación en memoria física:

- la ubicación contiene un valor, el cual puede cambiar a lo largo del tiempo por efecto de comandos de asignación



Variable

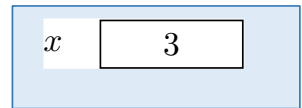
En matemáticas, un nombre que representa un valor, tomado de algún dominio predeterminado

En lenguajes de programación, sigue siendo un nombre que representa un objeto ... pero además ...

Variable modificable (en el modelo de valor)

Nombre que representa una parte de almacenamiento modificable, una suerte de contenedor, o ubicación en memoria física:

- la ubicación contiene un valor, el cual puede cambiar a lo largo del tiempo por efecto de comandos de asignación

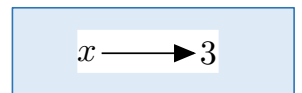


Versión abstracta de un puntero

Variable en el modelo de referencia

Nombre que representa una *referencia* (un mecanismo que da acceso) a un valor:

- p.ej., Python, Scala, Java (sólo para el caso de clases)



La **asignación** es el comando básico para la “modificación de una variable”:
→ cambiar el valor (o el contenido) asociado con un nombre

Comando más elemental
en todo lenguaje imperativo

La *asignación* es el comando básico para la “modificación de una variable”:

→ cambiar el valor (o el contenido) asociado con un nombre

P.ej. (en Pascal):

$X := 2$

$X := X + 1$

Comando más elemental
en todo lenguaje imperativo

La *asignación* es el comando básico para la “modificación de una variable”:

→ cambiar el valor (o el contenido) asociado con un nombre

El contenedor asociado con la variable (cuyo nombre es) *X* contiene a partir de ahora el valor 2

El comando no retorna ningún valor

→ el efecto de la asignación es un “efecto lateral”:

- a partir de ahora, el uso de *X* retornará el valor 2
- el valor previo almacenado en el contenedor se pierde

P.ej. (en Pascal):

```
X := 2  
X := X + 1
```

Comando más elemental
en todo lenguaje imperativo

La *asignación* es el comando básico para la “modificación de una variable”:

→ cambiar el valor (o el contenido) asociado con un nombre

El mismo nombre, X, se usa con significados diferentes a cada lado del operador de asignación :=

P.ej. (en Pascal):

`X := 2`

`X := X + 1`

Al lado izquierdo, indica el contenedor (o la ubicación) que se va a usar para almacenar el nuevo valor

Al lado derecho, representa el valor dentro del contenedor

Comando más elemental
en todo lenguaje imperativo

La *asignación* es el comando básico para la “modificación de una variable”:

→ cambiar el valor (o el contenido) asociado con un nombre

Dada una expresión *expr* que aparece en una asignación

... se habla del

l-value de *expr*, cuando *expr* aparece al lado izquierdo del operador de asignación, que es un valor que indica ubicación

r-value de *expr*, cuando *expr* aparece al lado derecho del operador de asignación, que es un valor que puede ser almacenado en una ubicación

P.ej. (en Pascal):

$X := 2$
 $X := X + 1$

Se calcula el *r-value* de 2 o de $X + 1$

En ambos casos, se calcula el *l-value* de X

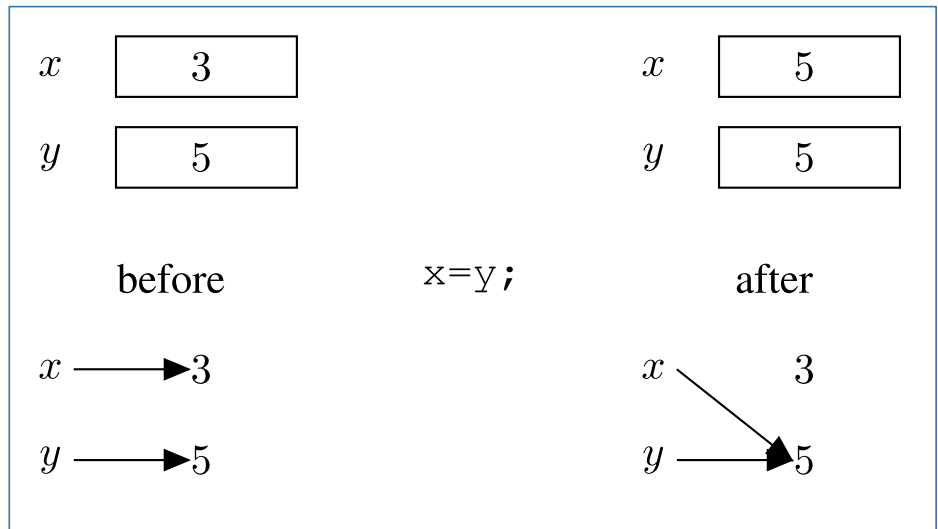
Diferencia de la semántica de la asignación entre **variables modificables** y variables en el modelo de referencia

1. Calcular el *l-value* de *x*, determinando un contenedor *loc*
2. Calcular el *r-value* de *y*, obteniendo un valor *val*
3. Cambiar el contenido de *loc* sustituyendo *val* por el que había previamente

<i>x</i>	3		<i>x</i>	5
<i>y</i>	5		<i>y</i>	5
before		$x=y;$	after	

Diferencia de la semántica de la asignación entre variables modificables y **variables en el modelo de referencia**

1. Calcular el l-value de x , determinando un contenedor loc
2. Calcular el r-value de y , obteniendo un valor val
3. Cambiar el contenido de loc sustituyendo val por el que había previamente



1. Calcular el l -value de x , determinando una referencia ref
2. Calcular el r -value de y , obteniendo un valor val
3. Cambiar la referencia ref , de modo que ahora se refiera a val ; val no es copiado (duplicado)

La asignación es el comando básico: paso computacional elemental

Los otros comandos definen el control de la secuencia:

- especifican el orden en el cual deben ser ejecutadas los cambios al estado producidos por las asignaciones

La asignación es el comando básico: paso computacional elemental

Los otros comandos definen el control de la secuencia:

- especifican el orden en el cual deben ser ejecutadas los cambios al estado producidos por las asignaciones

Comandos para control explícito de secuencia:

- El comando secuencial y el goto; también, el comando compuesto: un grupo de comandos es considerado como un comando individual

Comandos condicionales (o de selección):

- Permiten la especificación de rutas alternativas que puede tomar el flujo, dependiendo de los valores de condiciones

Comandos iterativos:

- Permiten que un determinado comando sea repetido un predeterminado número de veces o hasta que se cumplan ciertas condiciones

La asignación es el comando básico: paso computacional elemental

Los otros comandos definen el control de la secuencia:

- especifican el orden en el cual deben ser ejecutadas los cambios al estado producidos por las asignaciones

Comandos para control explícito de secuencia:

- El comando secuencial y el goto (además de *break*, *continue*, *return*); también, el comando compuesto (*bloque*): un grupo de comandos que es considerado como un comando individual

Comandos condicionales (o de selección):

- Permiten la especificación de rutas alternativas que puede tomar el flujo, dependiendo de los valores de condiciones

Comandos iterativos:

- Permiten que un determinado comando sea repetido un predeterminado número de veces o hasta que se cumplan ciertas condiciones

El comando *secuencial* es especificado en varios lenguajes por un “;”: C1 ; C2

El comando *compuesto* es especificado por delimitadores: *begin ... end* , { ... } ; o por indentación

goto inspirado en instrucciones *jump* de lenguajes *assembly*

Comandos condicionales:

- if
- case

```
if Bexp then C1 else C2
if Bexp then C1
if Bexp1 then C1
  elseif Bexp2 then C2
  ...
  elseif Bexpn then Cn
  else Cn+1
endif
```

```
case Exp of
  label1: C1;
  label2: C2;
  ...
  labeln: Cn;
else Cn+1
```

```
switch (Exp) {
  case label1: C1 break;
  case label2: C2 break;
  ...
  case labeln: Cn break;
  default Cn+1 break;
}
```

```
if x = 0:  
    hacer algo  
else:  
    hacer otra cosa  
...
```

instrucciones en *assembly*: la versión en lenguaje de máquina está en la *Instruction Memory* (cada línea va en una dirección diferente)

```
CMP A,0  
JNE else  
...  
...  
...  
JMP end  
else:  
...  
...  
...  
end:  
...
```

código *assembly* correspondiente a *hacer algo*

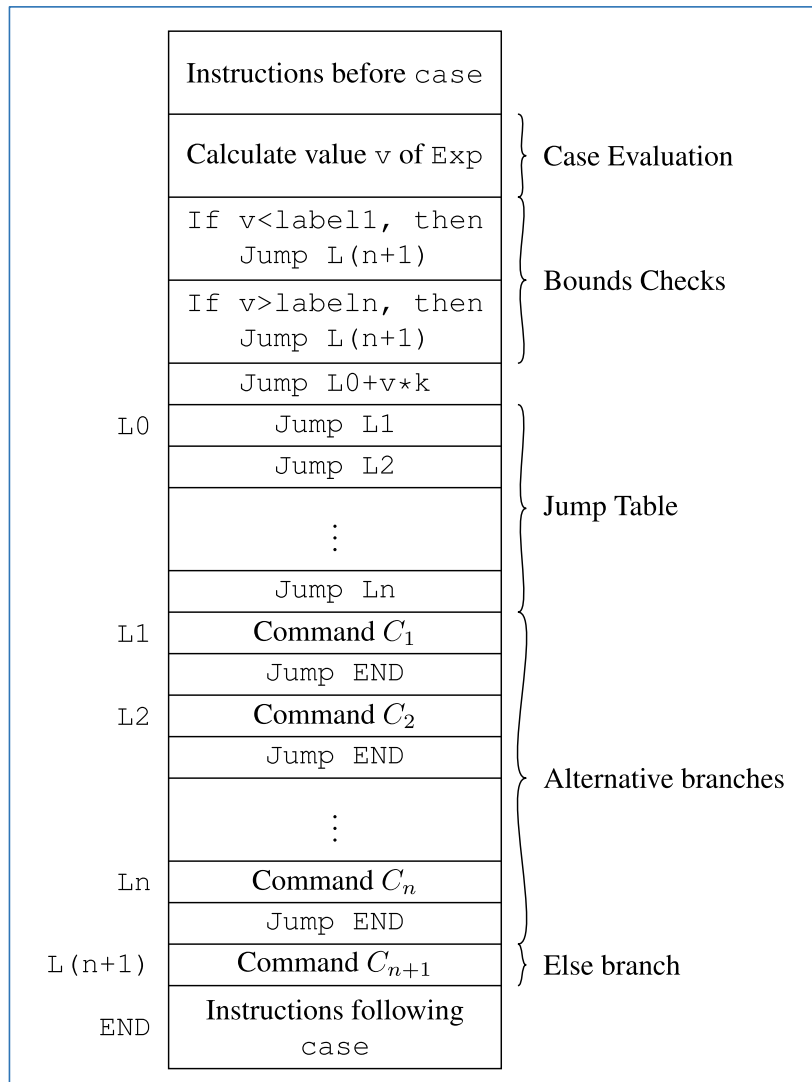
código *assembly* correspondiente a *hacer otra cosa*

labels: corresponden a las direcciones de las instrucciones

```

case Exp of
  label1: C1;
  label2: C2;
  ...
  labeln: Cn;
else Cn+1

```



Todos los comandos anteriores (excepto `goto`) permiten expresar sólo computaciones finitas

(... cuya longitud máxima está determinada estáticamente por la longitud del texto del programa)

- p.ej., no permiten procesar un vector de n elementos, en que n no se conoce a prior

En lenguajes de bajo nivel, la capacidad de expresar todos los algoritmos posibles se consigue mediante las instrucciones `jump`

En lenguajes de alto nivel:

- **iteración** (estructurada) limitada (controlada numéricamente) o ilimitada (controlada lógicamente)
- **recursión**

Iteración ilimitada:

`while (Bexp) do C`

`repeat C until Bexp`

`do C while (Bexp)`

`C;`
`while (not Bexp) do C`

`C;`
`while (Bexp) do C`

Simple de implementar

→ corresponde directamente a un *loop* que es implementado en hardware usando una instrucción *jump* condicional

instrucciones en *assembly*: la versión en lenguaje de máquina está en la *Instruction Memory* (cada línea va en una dirección diferente)

```
while i > 0:  
    hacer algo  
    i = i-1  
...
```

```
while: CMP A,0  
       JLE end  
       ...  
       ...  
       ...  
       SUB A,1  
       JMP while  
end:   ...
```

labels: corresponden a las direcciones de las instrucciones

código *assembly* correspondiente a *hacer algo*

Recursión

Un subprograma es recursivo si su cuerpo contiene una llamada a sí mismo

Recursión indirecta, o *mutua*: Un subprograma *P* llama a otro, *Q*, el cual a su vez llama a *P*

Recursión

Mecanismo alternativo a la iteración (ilimitada) para que un lenguaje sea *Turing complete*

Un subprograma es recursivo si su cuerpo contiene una llamada a sí mismo

Recursión indirecta, o *mutua*: Un subprograma *P* llama a otro, *Q*, el cual a su vez llama a *P*

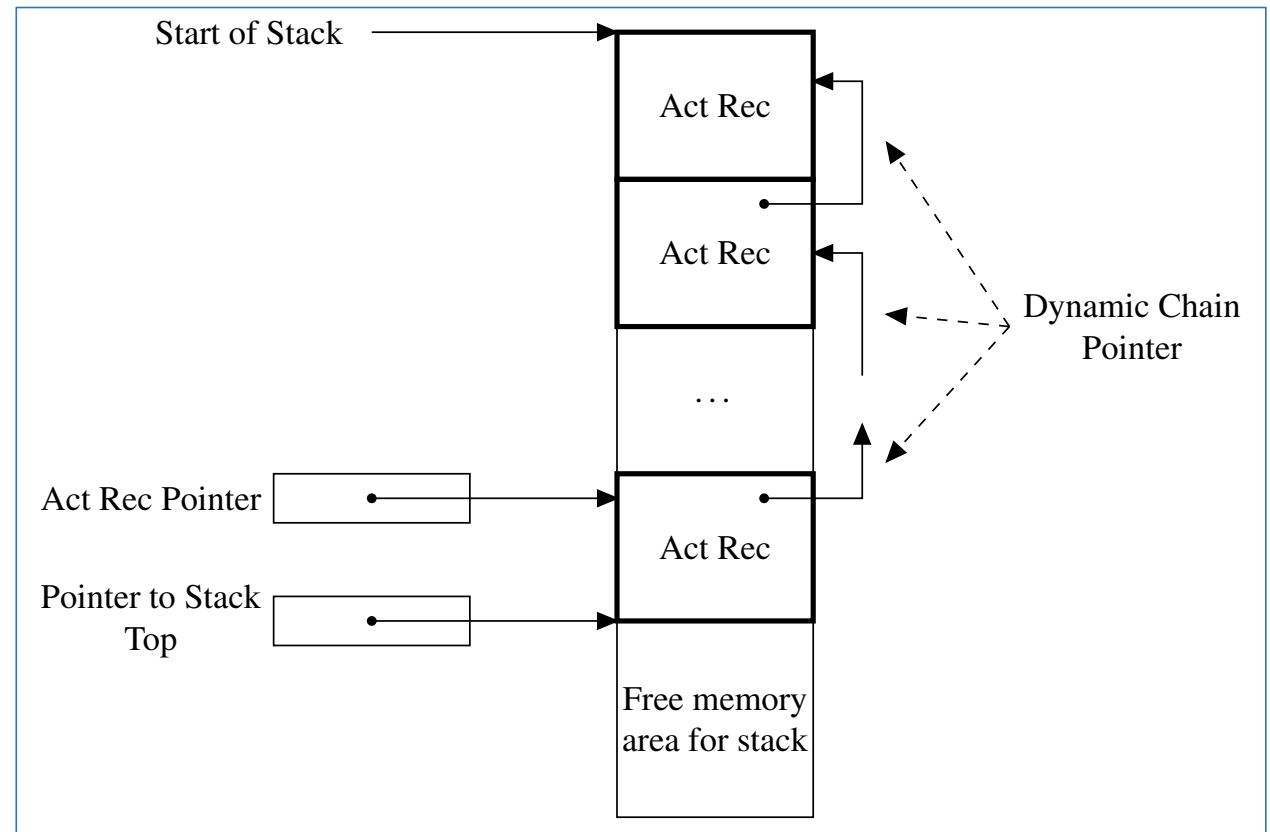
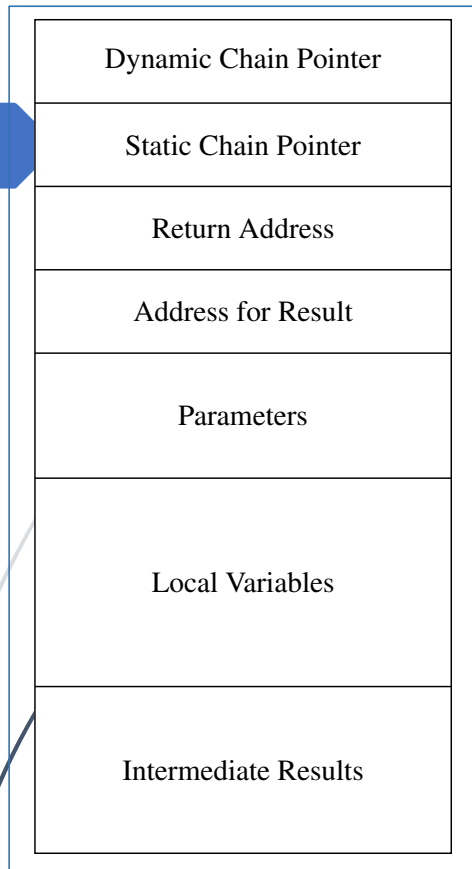
Recursión

Mecanismo alternativo a la iteración (ilimitada) para que un lenguaje sea *Turing complete*

Un subprograma es recursivo si su cuerpo contiene una llamada a sí mismo

Recursión indirecta, o *mutua*: Un subprograma P llama a otro, Q , el cual a su vez llama a P

En matemáticas, las definiciones inductivas permiten describir el resultado de la aplicación de una función f a un argumento X en términos de la aplicación de la propia f a argumentos que sean “más pequeños” que X



La recursión hace necesario incluir manejo dinámico de memoria
→ no es posible determinar estáticamente el número máximo de instancias de una misma función que estarán activas al mismo tiempo

P.ej., ...

```
int fact (int n){  
    if (n <= 1)    return 1;  
    else  
        return n * fact(n-1);  
}
```

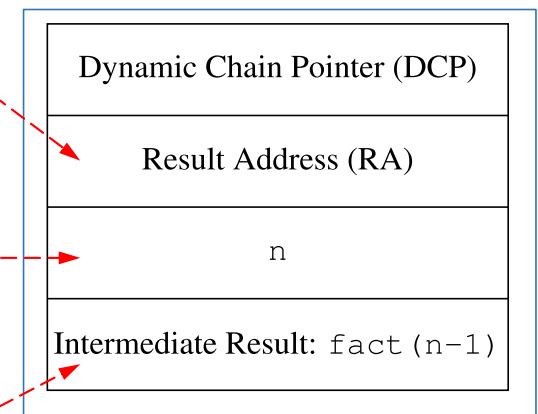

P.ej., ...

```
int fact (int n){  
    if (n <= 1) return 1;  
    else  
        return n * fact(n-1);  
}
```

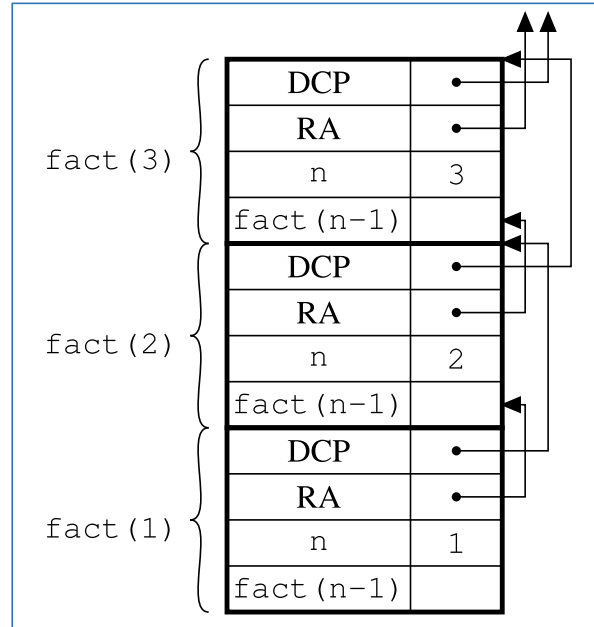
Dirección del campo
Intermediate Result en
el RA del que llama

Parámetro actual

Resultado de la eva-
luación de `fact(n-1)`

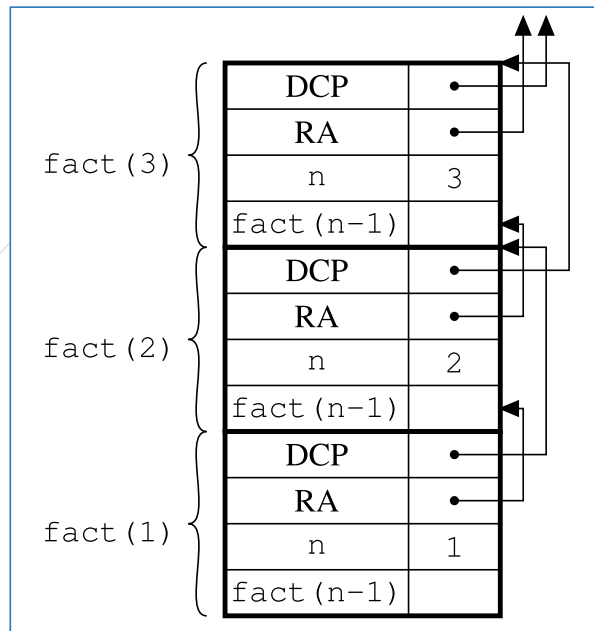


Registro de activa-
ción (**RA**) de `fact`



El valor del campo *Intermediate Result* en el RA de `fact(n)` puede determinarse **sólo** cuando la llamada recursiva a `fact(n-1)` termina

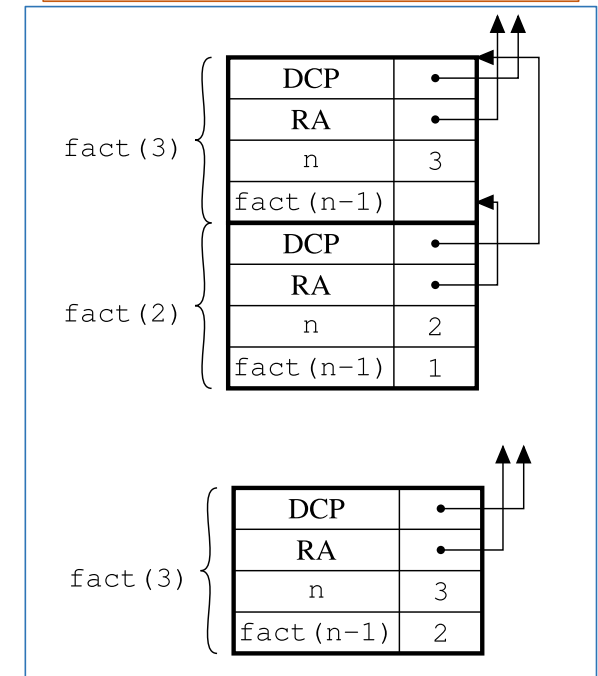
... este valor se usa para calcular $n * \text{fact}(n-1)$, el valor de `fact(n)`
 $\Rightarrow$ todos los RAs para las llamadas recursivas `fact(n)`, `fact(n-1)`, ..., `fact(1)`, deben estar en el stack al mismo tiempo, en áreas de memoria diferentes



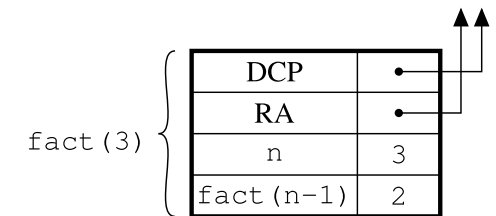
El valor del campo *Intermediate Result* en el RA de $\text{fact}(n)$ puede determinarse **sólo** cuando la llamada recursiva a $\text{fact}(n-1)$ termina

... este valor se usa para calcular $n * \text{fact}(n-1)$, el valor de $\text{fact}(n)$
 $\Rightarrow$ todos los RAs para las llamadas recursivas $\text{fact}(n)$, $\text{fact}(n-1)$, ..., $\text{fact}(1)$, deben estar en el stack al mismo tiempo, en áreas de memoria diferentes

Stack de RAs después de terminar la llamada a $\text{fact}(1)$



Stack de RAs después de terminar la llamada a $\text{fact}(2)$



Sin embargo, ...

```
int factrc (int n, int res){  
    if (n <= 1)  
        return res;  
    else  
        return factrc(n-1, n * res)  
}
```

La llamada `factrc(n,1)`
devuelve el factorial de n

Sin embargo, ...

37

```
int factrc (int n, int res){  
    if (n <= 1)  
        return res;  
    else  
        return factrc(n-1, n * res)  
}
```

La llamada `factrc(n,1)`
devuelve el factorial de n

La llamada inicial `factrc(n, 1)` produce $n - 1$ llamadas recursivas:

`factrc(n-1, n*1)`

`factrc(n-2, (n-1)*n*1)`

...

`factrc(1, 2*...*(n-1)*n*1)`

Pero ... el valor devuelto por `factrc(n,res)` es exactamente igual al valor devuelto por `factrc(n-1,n*res)`

⇒ Cuando `factrc(n,res)` ha llamado a `factrc(n-1, n*res)`, no es más necesario mantener información en el RA sobre `factrc(n,res)`

→ la información necesaria para calcular el resultado final es pasada a `factrc(n-1,n*res)`

⇒ el RA para `factrc(n-1,n*res)` puede reusar la memoria asignada al RA de `factrc(n,res)`

Sin embargo, ...

38

```
int factrc (int n, int res){  
    if (n <= 1)  
        return res;  
    else  
        return factrc(n-1, n * res)  
}
```

La llamada `factrc(n,1)`
devuelve el factorial de n

Lo mismo para todas las llamadas sucesivas
→ `factrc` necesita una única área de memoria
para almacenar un único RA, independiente
de cuántas llamadas recursivas se hagan
→ una función recursiva para la cual la memo-
ria se puede asignar estáticamente

La llamada inicial `factrc(n, 1)` produce $n - 1$
llamadas recursivas:

`factrc(n-1, n*1)`

`factrc(n-2, (n-1)*n*1)`

...

`factrc(1, 2*...*(n-1)*n*1)`

Pero ... el valor devuelto por `factrc(n,res)` es
exactamente igual al valor devuelto por
`factrc(n-1,n*res)`

⇒ Cuando `factrc(n,res)` ha llamado a
`factrc(n-1, n*res)`, no es más necesario mante-
ner información en el RA sobre `factrc(n,res)`

→ la información necesaria para calcular el
resultado final es pasada a `factrc(n-1,n*res)`

⇒ el RA para `factrc(n-1,n*res)` puede reusar la
memoria asignada al RA de `factrc(n,res)`

Sea f una función que contiene una llamada a una función g
La llamada a g es una *llamada de cola* si f retorna el valor
retornado por g sin hacer ninguna computación adicional
 f es *recursiva de cola* si todas las llamadas recursivas en f son
llamadas de cola

Sea f una función que contiene una llamada a una función g

La llamada a g es una *llamada de cola* si f retorna el valor retornado por g sin hacer ninguna computación adicional

f es *recursiva de cola* si todas las llamadas recursivas en f son llamadas de cola

```
int factrc (int n, int res){  
    if (n <= 1)  
        return res;  
    else  
        return factrc(n-1, n * res)  
}
```

factrc es recursiva de cola: La llamada recursiva es lo último que pasa en el cuerpo de factrc ... después de la cual no se ejecuta ninguna otra computación

Sea f una función que contiene una llamada a una función g

La llamada a g es una *llamada de cola* si f retorna el valor retornado por g sin hacer ninguna computación adicional

f es *recursiva de cola* si todas las llamadas recursivas en f son llamadas de cola

```
int factrc (int n, int res){  
    if (n <= 1)  
        return res;  
    else  
        return factrc(n-1, n * res)  
}
```

factrc es recursiva de cola: La llamada recursiva es lo último que pasa en el cuerpo de factrc ... después de la cual no se ejecuta ninguna otra computación

```
int f (int n){  
    if (n == 0)  
        return 1;  
    else  
        if (n == 1)  
            return f(0);  
        else  
            if (n == 2)  
                return f(n-1);  
            else  
                return f(1) * 2;  
}
```

Llamada de cola

Llamada de cola

f no es recursiva de cola

No es llamada de cola

Ventaja de la recursión de cola → puede implementarse usando un único RA → espacio de memoria constante

Siempre podemos transformar una definición de una función que no es recursiva de cola en otra que sí lo es:

- lo más posible de la computación que se hace después de la llamada recursiva debe hacerse antes de llamada
- lo que no pueda hacerse antes, se “pasa” mediante parámetros adicionales a la propia llamada recursiva

Ventaja de la recursión de cola → puede implementarse usando un único RA → espacio de memoria constante

Siempre podemos transformar una definición de una función que no es recursiva de cola en otra que sí lo es:

- lo más posible de la computación que se hace después de la llamada recursiva debe hacerse antes de llamada
- lo que no pueda hacerse antes, se “pasa” mediante parámetros adicionales a la propia llamada recursiva

En `factrc`, en lugar de calcular $n \cdot \text{fact}(n-1)$ en el cuerpo de la llamada a `fact(n-1)`

... agregamos el parámetro `res`: pasamos el producto $n \cdot (n-1) \cdot (n-2) \cdots j$ a la llamada `factrc(j-1, j*res)`

→ el cálculo del factorial se hace incrementalmente por las sucesivas llamadas recursivas

Ventaja de la recursión de cola → puede implementarse usando un único RA → espacio de memoria constante

Siempre podemos transformar una definición de una función que no es recursiva de cola en otra que sí lo es:

- lo más posible de la computación que se hace después de la llamada recursiva debe hacerse antes de llamada
- lo que no pueda hacerse antes, se “pasa” mediante parámetros adicionales a la propia llamada recursiva

En `factrc`, en lugar de calcular $n \cdot \text{fact}(n-1)$ en el cuerpo de la llamada a `fact(n-1)`

... agregamos el parámetro `res`: pasamos el producto $n \cdot (n-1) \cdot (n-2) \dots j$ a la llamada `factrc(j-1, j*res)`

→ el cálculo del factorial se hace incrementalmente por las sucesivas llamadas recursivas

... análogamente a una versión iterativa

```
int fact-it (int n, int res){  
    res=1;  
    for (i=n; i>=1; i--)  
        res = res*i;  
}
```

Similarmente, con la función `fib`, que calcula el n -ésimo valor de la serie de Fibonacci

```
int fib (int n){  
    if (n == 0)  
        return 1;  
    else  
        if (n == 1)  
            return 1;  
        else  
            return fib(n-1) + fib(n-2);  
}
```

```
int fibrc (int n, int res1, int res2){  
    if (n == 0)  
        return res2;  
    else  
        if (n == 1)  
            return res2;  
        else  
            return fibrc(n-1, res2, res1+res2);  
}
```

La llamada inicial
es `fibrc(n,1,1)`

Se dice que la iteración es más eficiente que la recursión

Como hemos visto, es más exacto decir que una implementación simple de la iteración es más eficiente que una implementación simple de la recursión

Un compilador “optimizante” puede generar código de excelente calidad para funciones recursivas, en especial para funciones recursivas de cola

Se dice que la iteración es más eficiente que la recursión

Como hemos visto, es más exacto decir que una implementación simple de la iteración es más eficiente que una **implementación simple de la recursión**

Un compilador “optimizante” puede generar código de excelente calidad para funciones recursivas, en especial para funciones recursivas de cola

Traducción directa de la definición inductiva:

- hace efectivamente llamadas a subrutinas que asignan espacio en el stack de *run time* para variables locales e información adicional

Se dice que la iteración es más eficiente que la recursión

Como hemos visto, es más exacto decir que una implementación simple de la iteración es más eficiente que una **implementación simple de la recursión**

Un compilador “optimizante” puede generar código de excelente calidad para funciones recursivas, en especial para funciones recursivas de cola

Traducción directa de la definición inductiva:

- hace efectivamente llamadas a subrutinas que asignan espacio en el stack de *run time* para variables locales e información adicional

```
int gcd(int a, int b) {  
    /* assume a, b > 0 */  
    if (a == b) return a;  
    else if (a > b) return gcd(a-b, b);  
    else return gcd(a, b-a);  
}
```



```
int gcd(int a, int b) {  
    /* assume a, b > 0 */  
start:  
    if (a == b) return a;  
    else if (a > b) {  
        a = a-b; goto start;  
    } else {  
        b = b-a; goto start;  
    }  
}
```


Al escribir un programa
... ¿iteración o recursión?

Manejo de datos que usan estructuras rígidas, en aplicaciones numéricas o de procesamiento de datos:

- matrices, tablas, ...

Procesamiento de estructuras simbólicas que se prestan naturalmente para definiciones recursivas:

- listas, árboles, ...

Programación estructurada

(Rechazo al goto)

Serie de prescripciones → Desarrollo de software con una cierta estructura en el código y en el flujo de control

- componente metodológica: cómo desarrollar programas
- componente lingüística: los lenguajes debería preferir ciertos tipos de comandos

Diseño jerárquico (top-down) de programas

Modularización del código

Uso de nombres significativos

Uso de comentarios

Uso de tipos de datos estructurados

Uso de construcciones de control estructuradas:

- un sólo punto de entrada y un sólo punto de salida



Una revisión lineal del texto del programa corresponde al flujo de ejecución:

- si el comando C2 sigue textualmente al comando C1, a la salida única de C1, cuando C1 termina, el control pasa a la entrada única de C2

Programación estructurada

(Rechazo al goto++)

Serie de prescripciones → Desarrollo de software con una cierta estructura en el código y en el flujo de control

- componente metodológica: cómo desarrollar programas
- componente lingüística: los lenguajes deberían preferir ciertos tipos de comandos

Un primer paso hacia los métodos modernos de programación

Programación estructurada

(Rechazo al goto++)

Serie de prescripciones → Desarrollo de software con una cierta estructura en el código y en el flujo de control

- componente metodológica: cómo desarrollar programas
- componente lingüística: los lenguajes deberían preferir ciertos tipos de comandos

Programación estructurada

(Rechazo al goto++)

Serie de prescripciones → Desarrollo de software con una cierta estructura en el código y en el flujo de control

- componente metodológica: cómo desarrollar programas
- componente lingüística: los lenguajes deberían preferir ciertos tipos de comandos

Diseño jerárquico (*top-down*) de programas, mediante refinaciones sucesivas

Modularización del código, agrupando juntos todos los comandos que implementan una funcionalidad específica:

- desde comandos compuestos hasta módulos y subprogramas

Uso de **nombres significativos** para variables, subprogramas, etc.:

- para facilitar la comprensión del código

Uso de **comentarios**

Uso de **tipos de datos estructurados**, para agrupar y estructurar información, en especial, heterogénea

- p.ej., records, clases

...

...

Uso de **construcciones de control estructuradas**,
i.e., con un único punto de entrada y un único
punto de salida

P.ej., asignaciones, comandos condicionales
e iteración ilimitada

... que, en conjunto, definen un lenguaje
Turing complete

...

Uso de **construcciones de control estructuradas**, i.e., con un único punto de entrada y un único punto de salida

P.ej., asignaciones, comandos condicionales e iteración ilimitada

... que, en conjunto, definen un lenguaje *Turing complete*

Permiten estructurar el código de manera que una revisión lineal del texto del programa corresponde al flujo de ejecución:

- si el comando C2 sigue textualmente al comando C1, entonces a la salida única de C1, cuando C1 termina, el control pasa a la entrada única de C2
- C1 y C2 pueden tener estructura *interna* compleja, e.g., *ifs* o *loops*, pero externamente cada una es visible en términos sólo de un punto entrada y uno de salida

...

Uso de **construcciones de control estructuradas**, i.e., con un único punto de entrada y un único punto de salida

P.ej., asignaciones, comandos condicionales e iteración ilimitada

... que, en conjunto, definen un lenguaje *Turing complete*

Permiten estructurar el código de manera que una revisión lineal del texto del programa corresponde al flujo de ejecución:

- si el comando C2 sigue textualmente al comando C1, entonces a la salida única de C1, cuando C1 termina, el control pasa a la entrada única de C2
- C1 y C2 pueden tener estructura *interna* compleja, e.g., *ifs* o *loops*, pero externamente cada una es visible en términos sólo de un punto entrada y uno de salida

Propiedad incompatible con la presencia del **goto**:

- permite saltos hacia adelante y hacia atrás
- rápidamente, "código spaghetti"

Subprogramas (procedimientos, funciones)

Soporte lingüístico —i.e., instrumentos reales más allá de sugerencias metodológicas— provisto por los lenguajes de programación

... facilita y posibilita la subdivisión de un problema en subproblemas —lo que permite un mejor manejo de la complejidad: cada subproblema es más fácil de resolver

... y la construcción de la solución al problema original a partir de la composición apropiada de las soluciones a los subproblemas

Subprogramas (procedimientos, funciones)

Soporte lingüístico —i.e., instrumentos reales más allá de sugerencias metodológicas— provisto por los lenguajes de programación

... facilita y posibilita la subdivisión de un problema en subproblemas —lo que permite un mejor manejo de la complejidad: cada subproblema es más fácil de resolver

... y la construcción de la solución al problema original a partir de la composición apropiada de las soluciones a los subproblemas

Subprograma (procedimiento, función)

Pieza de código identificada por nombre

... tiene su propio *entorno* local

... puede intercambiar información con el resto del programa

- usando parámetros
- retornando un valor
- refiriéndose a un entorno no local

Dos mecanismos lingüísticos distintos:

- definición o declaración de la función
- uso o llamada

Subprograma (procedimiento, función)

Pieza de código identificada por nombre

... tiene su propio *entorno* local

... puede intercambiar información con el resto del programa

- usando parámetros
- retornando un valor
- refiriéndose a un entorno no local

Dos mecanismos lingüísticos distintos:

- definición o declaración de la función
- uso o llamada

foo

```
int foo (int n, int a) {  
    int tmp=a;  
    if (tmp==0) return n;  
    else return n+1;  
}  
...  
int x;  
x = foo(3,0);  
x = foo(x+1,1);
```

Subprograma (procedimiento, función)

Pieza de código identificada por nombre

... tiene su propio *entorno* local

... puede intercambiar información con el resto del programa

- usando parámetros
- retornando un valor
- refiriéndose a un entorno no local

Dos mecanismos lingüísticos distintos:

- definición o declaración de la función
- uso o llamada

Entorno (de referencia): Conjunto de asociaciones entre nombres y objetos denotables que existen en tiempo de ejecución en un punto específico en el programa y en un momento específico durante la ejecución

Subprograma (procedimiento, función)

Pieza de código identificada por nombre

... tiene su propio *entorno* local

... puede intercambiar información con el resto del programa

- usando parámetros
- retornando un valor
- refiriéndose a un entorno no local

Dos mecanismos lingüísticos distintos:

- definición o declaración de la función
- uso o llamada

n, a, tmp

```
int foo (int n, int a) {  
    int tmp=a;  
    if (tmp==0) return n;  
    else return n+1;  
}  
...  
int x;  
x = foo(3,0);  
x = foo(x+1,1);
```

Subprograma (procedimiento, función)

Pieza de código identificada por nombre

... tiene su propio *entorno* local

... puede intercambiar información con el resto del programa

- usando parámetros
- retornando un valor
- refiriéndose a un entorno no local

Dos mecanismos lingüísticos distintos:

- definición o declaración de la función
- uso o llamada

foo

n, a, tmp

```
int foo (int n, int a) {  
    int tmp=a;  
    if (tmp==0) return n;  
    else return n+1;  
}  
...  
int x;  
x = foo(3,0);  
x = foo(x+1,1);
```

Parámetros formales: Nombres que, desde el punto de vista del entorno, se comportan como declaraciones locales a la función

```
int foo (int n, int a) {  
    int tmp=a;  
    if (tmp==0) return n;  
    else return n+1;  
}  
...  
int x;  
x = foo(3,0);  
x = foo(x+1,1);
```


Parámetros formales: Nombres que, desde el punto de vista del entorno, se comportan como declaraciones locales a la función

Son *variables vinculadas*: Su renombramiento consistente no tiene efecto sobre la semántica de la función

- estas funciones `foo` son indistinguibles

```
int foo (int m, int b){  
  int tmp=b;  
  if (tmp==0) return m;  
  else return m+1;  
}
```

```
int foo (int n, int a) {  
  int tmp=a;  
  if (tmp==0) return n;  
  else return n+1;  
}  
...  
int x;  
x = foo(3,0);  
x = foo(x+1,1);
```

Parámetros formales: Nombres que, desde el punto de vista del entorno, se comportan como declaraciones locales a la función

Son *variables vinculadas*: Su renombramiento consistente no tiene efecto sobre la semántica de la función

- estas funciones `foo` son indistinguibles

```
int foo (int m, int b){  
  int tmp=b;  
  if (tmp==0) return m;  
  else return m+1;  
}
```

```
int foo (int n, int a) {  
  int tmp=a;  
  if (tmp==0) return n;  
  else return n+1;  
}  
...  
int x;  
x = foo(3,0);  
x = foo(x+1,1);
```

Parámetros actuales: Cantidad y tipo deben, en general, coincidir con los parámetros formales

Disciplina del paso de parámetros

El *modo* en el cual los parámetros actuales son emparejados con los parámetros formales

... y la semántica resultante

Modo:

- tipo de comunicación que permite
... + implementación que la produce
- queda fijo cuando la función es definida y puede ser diferente para cada parámetro

Disciplina del paso de parámetros

El modo en el cual los parámetros actuales son emparejados con los parámetros formales

... y la semántica resultante

Modo:

- tipo de comunicación que permite

... + implementación que la produce

- queda fijo cuando la función es definida y puede ser diferente para cada parámetro

- Por valor (+ variaciones)
- Por referencia
- Por nombre

Disciplina del paso de parámetros

El modo en el cual los parámetros actuales son emparejados con los parámetros formales

... y la semántica resultante

Modo:

- tipo de comunicación que permite ... + implementación que la produce
- queda fijo cuando la función es definida y puede ser diferente para cada parámetro

- Por valor (+ variaciones)
- Por referencia
- Por nombre

- Parámetros de input
- Parámetros de output
- Parámetros de input/output

Disciplina del paso de parámetros

El modo en el cual los parámetros actuales son emparejados con los parámetros formales

... y la semántica resultante

Modo:

• tipo de comunicación que permite

... + implementación que la produce

• queda fijo cuando la función es definida y puede ser diferente para cada parámetro

- Por valor (+ variaciones)
- Por referencia
- Por nombre

- Parámetros de input
- Parámetros de output
- Parámetros de input/output

Para cada modo:

- ¿Qué tipo de comunicación permite?
- ¿Qué forma de parámetros actuales permite?
- ¿Cuál es su semántica?
- ¿Cuál es la implementación canónica?
- ¿Cuánto cuesta?

Llamada **por valor**

Parámetro de input

Entorno local (de la función) es extendido: asociación entre el parámetro formal y una nueva variable

El parámetro actual puede ser una expresión

Cuando se produce la llamada: el parámetro actual es evaluado
→ se obtiene su *r-value* y se asocia con el parámetro formal

Durante la ejecución de la función: no hay vínculo entre el parámetro formal y el actual

Al retornar la función: el parámetro formal es destruido (así como el entorno local de la función)

Llamada **por valor**

Parámetro de input

Entorno local (de la función) es extendido: asociación entre el parámetro formal y una nueva variable

El parámetro actual puede ser una expresión

Cuando se produce la llamada: el parámetro actual es evaluado
→ se obtiene su *r-value* y se asocia con el parámetro formal

Durante la ejecución de la función: no hay vínculo entre el parámetro formal y el actual

Al retornar la función: el parámetro formal es destruido (así como el entorno local de la función)

y nunca cambia su valor:
permanece siempre en 1

```
int y = 1;  
void foo (int x) {  
    x = x+1;  
}  
...  
y = 1;  
foo(y+1);  
// here y = 1
```

En C, C++, Pascal y Java, no se indica explícitamente el modo por valor

x toma el valor inicial 2
(como efecto del paso
de parámetro); y luego, 3

Llamada **por referencia**
(también, *por variable*)

Parámetro de input/output

El parámetro actual *debe* ser una expresión con l-value

Al momento de la llamada, el entorno local de la función es extendido con una asociación entre el parámetro formal y este l-value (evaluado en ese momento)

Si el parámetro actual es una variable (caso más común), entonces el formal y el actual son dos nombres para la misma variable

Al retornar la función, se destruye la conexión entre el parámetro formal y el l-value del parámetro actual (junto con el entorno local de la función)

Comunicación bidireccional: cada cambio del parámetro formal es un cambio del parámetro actual

Llamada **por referencia**
(también, *por variable*)

Parámetro de input/output

El parámetro actual *debe* ser una expresión con l-value

Al momento de la llamada, el entorno local de la función es extendido con una asociación entre el parámetro formal y este l-value (evaluado en ese momento)

Si el parámetro actual es una variable (caso más común), entonces el formal y el actual son dos nombres para la misma variable

Al retornar la función, se destruye la conexión entre el parámetro formal y el l-value del parámetro actual (junto con el entorno local de la función)

Comunicación bidireccional: cada cambio del parámetro formal es un cambio del parámetro actual

x es un nombre para y
Incrementar x es, para todos
los efectos, incrementar y

```
int y = 0;  
void foo (reference int x) {  
    x = x+1;  
}  
y=0;  
foo(y);  
// here y = 1
```

Usamos, p.ej., el
modificador reference

Llamada **por referencia**

El parámetro actual puede ser una expresión cuyo *l-value* se determina al momento de la llamada

```
int[] V = new V[10];  
int i=0;  
void foo (reference int x) {  
    x = x+1;  
}  
...  
V[1] = 1;  
foo(V[i+1]);  
// here V[1] = 2
```

x es un nombre para v[1]
Incrementar x es incrementar v[1]

Llamada por valor

El parámetro formal corresponde a una ubicación en el RA de la función

→ se almacena el valor del parámetro actual, durante la ejecución de la función

Es un modo caro cuando el parámetro está vinculado a una estructura de datos grande

→ toda la estructura se copia en el parámetro formal

... pero el costo de acceso es mínimo

→ es el costo de acceso a una variable local

Mecanismo simple, con semántica clara,
default en muchos lenguajes, único modo en C, Java y Python

Llamada por valor

El parámetro formal corresponde a una ubicación en el RA de la función

→ se almacena el valor del parámetro actual, durante la ejecución de la función

Es un modo caro cuando el parámetro está vinculado a una estructura de datos grande

→ toda la estructura se copia en el parámetro formal

... pero el costo de acceso es mínimo

→ es el costo de acceso a una variable local

Mecanismo simple, con semántica clara, *default* en muchos lenguajes, único modo en C, Java y Python

Excepto si el parámetro formal no es modificado en la función:

- el parámetro puede pasarse eficientemente por referencia —modo *sólo lectura*:

Permite expresiones arbitrarias como parámetros actuales, sujetos a que no pueden ser modificados por la función, ni directa (via asignación) ni indirectamente (via llamadas a otras funciones)

Semánticamente, coincide con por valor

... pero la implementación queda a criterio del compilador:

- para datos de tamaño pequeño → por valor
- para datos de gran tamaño → por referencia

Llamada por valor

El parámetro formal corresponde a una ubicación en el RA de la función

→ se almacena el valor del parámetro actual, durante la ejecución de la función

Es un modo caro cuando el parámetro está vinculado a una estructura de datos grande

→ toda la estructura se copia en el parámetro formal

... pero el costo de acceso es mínimo

→ es el costo de acceso a una variable local

Mecanismo simple, con semántica clara, *default* en muchos lenguajes, único modo en C, Java y Python

C simula llamada por referencia manipulado punteros y direcciones:

- p.ej., una función que intercambia los valores de dos variables
- sólo con llamadas por valor, no hay cómo hacerlo

Llamada por valor

El parámetro formal corresponde a una ubicación en el RA de la función

→ se almacena el valor del parámetro actual, durante la ejecución de la función

Es un modo caro cuando el parámetro está vinculado a una estructura de datos grande

→ toda la estructura se copia en el parámetro formal

... pero el costo de acceso es mínimo

→ es el costo de acceso a una variable local

Mecanismo simple, con semántica clara, *default* en muchos lenguajes, único modo en C, Java y Python

C simula llamada por referencia manipulando punteros y direcciones:

- p.ej., una función que intercambia los valores de dos variables

→ con sólo llamada por valor, no hay cómo hacerlo

Los parámetros formales a intercambiar (ambos por valor) son de tipo puntero (a entero)

```
void swap (int *a, int *b) {  
    int tmp = *a; *a=*b; *b=tmp;  
}  
  
int v1 = ...;  
int v2 = ...;  
swap (&v1, &v2);
```

Toma el valor contenido en la ubicación cuya dirección está contenida en b y almacénalo en la ubicación cuya dirección está almacenada en a

Llamada por valor

El parámetro formal corresponde a una ubicación en el RA de la función

→ se almacena el valor del parámetro actual, durante la ejecución de la función

Es un modo caro cuando el parámetro está vinculado a una estructura de datos grande

→ toda la estructura se copia en el parámetro formal

... pero el costo de acceso es mínimo

→ es el costo de acceso a una variable local

Mecanismo simple, con semántica clara, *default* en muchos lenguajes, único modo en C, Java y Python

Llamada por referencia

Cada parámetro formal es asociado con una ubicación en el RA de la función:

- durante la llamada, el l-value del parámetro actual se almacena en el RA —una dirección
- el acceso al parámetro formal se hace via indirección usando (la dirección almacenada en) esta ubicación

⇒ Muy bajo costo

Llamada **por resultado**

Parámetro de output (dual de llamada por valor)

El argumento *debe* ser una expresión con *l-value*

El entorno local de la función es extendido con una asociación entre el parámetro (formal) y una nueva variable local

El argumento *debe* ser una expresión que evalúa a un *l-value*:

- al retornar la función, antes de la destrucción del entorno local, el valor vigente del parámetro es asignado a la ubicación especificada por este *l-value*

Durante la ejecución de la función, no hay ningún vínculo entre el parámetro y el argumento

Simplifica el diseño de funciones que deben devolver más de un valor, c/u en una variable diferente

```
void foo (result int x) {x = 8;}  
...  
int y = 1;  
foo(y);  
    // here y is 8
```

Llamada **por valor-resultado**

Parámetro de input/output

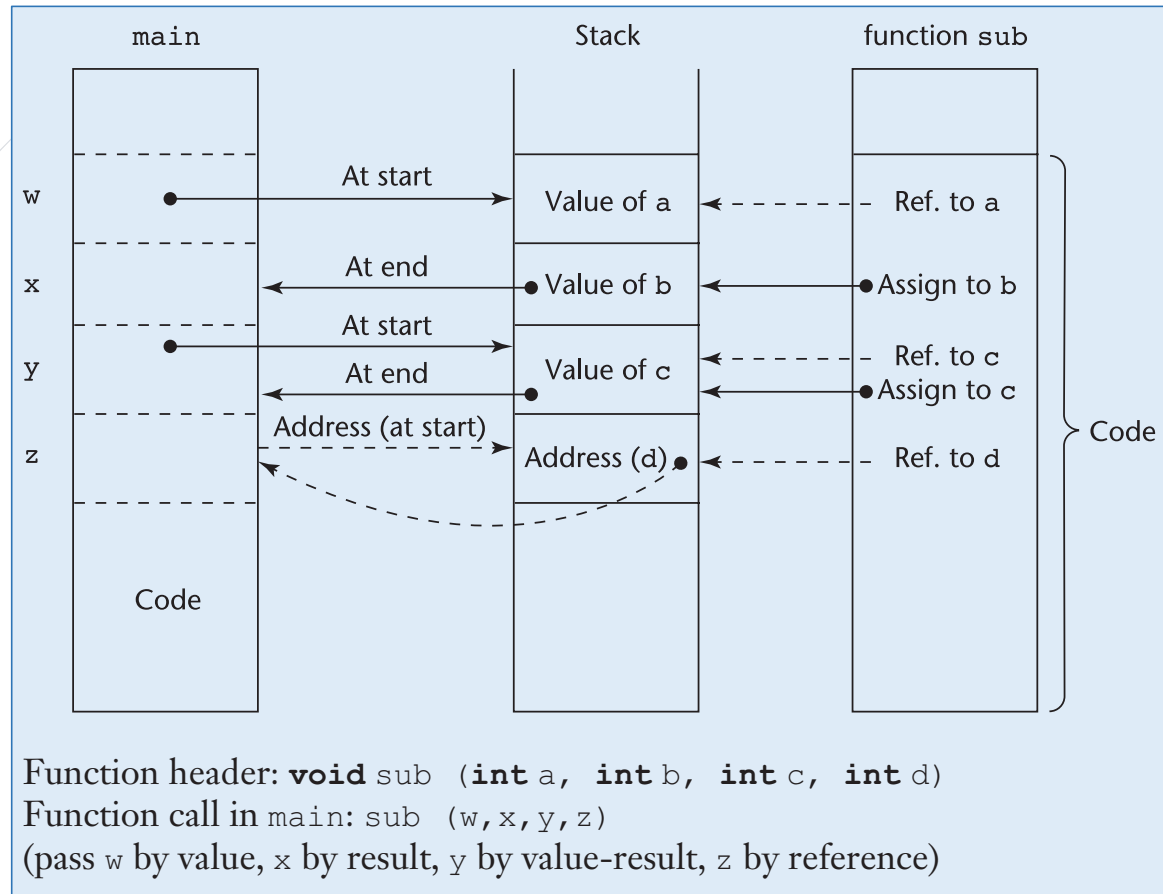
Combinación de llamada por valor y llamada por resultado

El parámetro (formal) es una variable local a la función

El argumento debe ser una expresión que evalúa a un *l-value*:

- al llamar a la función, el argumento es evaluado y su *r-value* es asignado al parámetro
- al retornar la función, antes de la destrucción del entorno local, el valor vigente del parámetro es asignado a la ubicación especificada por el *l-value*

```
void foo (valueresult int x){  
    x = x+1;  
}  
...  
y = 8;  
foo(y);  
    // here y is 9
```



Llamada **por valor-resultado**
no es llamada **por referencia**

```
void foo (reference/valueresult int x,  
         reference/valueresult int y,  
         reference int z){  
    y = 2;  
    x = 4;  
    if (x == y) z = 1;  
}  
...  
int a = 3;  
int b = 0;  
foo(a,a,b);
```

x y y tienen valores diferentes sólo en
ausencia de *aliasing*

- p.ej., pasadas por valor-resultado

Si, por el contrario, x y y son dos nombres
diferentes para la misma variable, enton-
ces `x == y` es siempre verdadero

- p.ej., pasadas por referencia

x y y pasadas por valor-resultado:
`foo(a,a,b)` termina sin modificar b

x y y pasadas por referencia:
`foo(a,a,b)` termina asignando 1 a b

Llamada **por nombre**

```
int x=0;
int foo (name int y){
    int z = 2;
    return z + y;
}
...
int a = foo(x+1)
```

→ `return z + x+1` → 3

Semánticamente, más limpia que por referencia

A partir de definir una semántica precisa para la definición de y llamada a funciones

- ¿Cuál debería ser el efecto de una llamada a una función?

Regla de la copia:

- Sea f una función con un solo parámetro (formal), x , y sea a una expresión de un tipo compatible con el de x . La llamada a f con argumento a es semánticamente equivalente a la ejecución del cuerpo de f en el cual todas las ocurrencias del parámetro x han sido reemplazadas por a

→ La operación sintáctica de expandir el cuerpo de la función después de una sustitución textual del parámetro

```
int x=0;  
int foo (name int y){  
    int z = 2;  
    return z + y;  
}  
...  
int a = foo(x+1)
```

→ return z + x+1 → 3

```
int x=0;  
int foo (name int y){  
    int x = 2;  
    return x + y;  
}  
...  
int a = foo(x+1);
```

→ return x + x+1 → 5

```
int x=0;  
int foo (name int y){  
    int z = 2;  
    return z + y;  
}  
...  
int a = foo(x+1)
```

return z + x+1

3

```
int x=0;  
int foo (name int y){  
    int x = 2;  
    return x + y;  
}  
...  
int a = foo(x+1);
```

El argumento x es *capturado*
por la declaración local

return x + x+1

5

La sustitución en la regla de la copia debe ser una que *no capture variables*

→ el parámetro (formal) debe ser evaluado en el entorno del que llama y no del que es llamado

→ lo que es sustituido no es sólo el argumento ... sino el argumento *junto* con su propio entorno de evaluación

... el cual queda determinado al momento de la llamada

La regla de la copia requiere que el argumento sea evaluado sólo cuando (y sólo si) el parámetro es encontrado durante la ejecución

... además, que sea evaluado cada vez que el parámetro sea encontrado durante la ejecución

- p.ej., supongamos que `i++` devuelve el valor actual de `i` y luego incrementa el valor de `i` en 1

```
int i = 2;
int fie (name int y) {
    return y+y;
}
...
int a = fie(i++);
    // here i has value 4; a has value 5
```

`i++` es evaluado dos veces (una por cada ocurrencia de `y`):

- la primera, devuelve 2 e `i` es incrementado a 3
- la segunda, devuelve 3 e `i` es incrementado a 4

Implementación de llamada por nombre

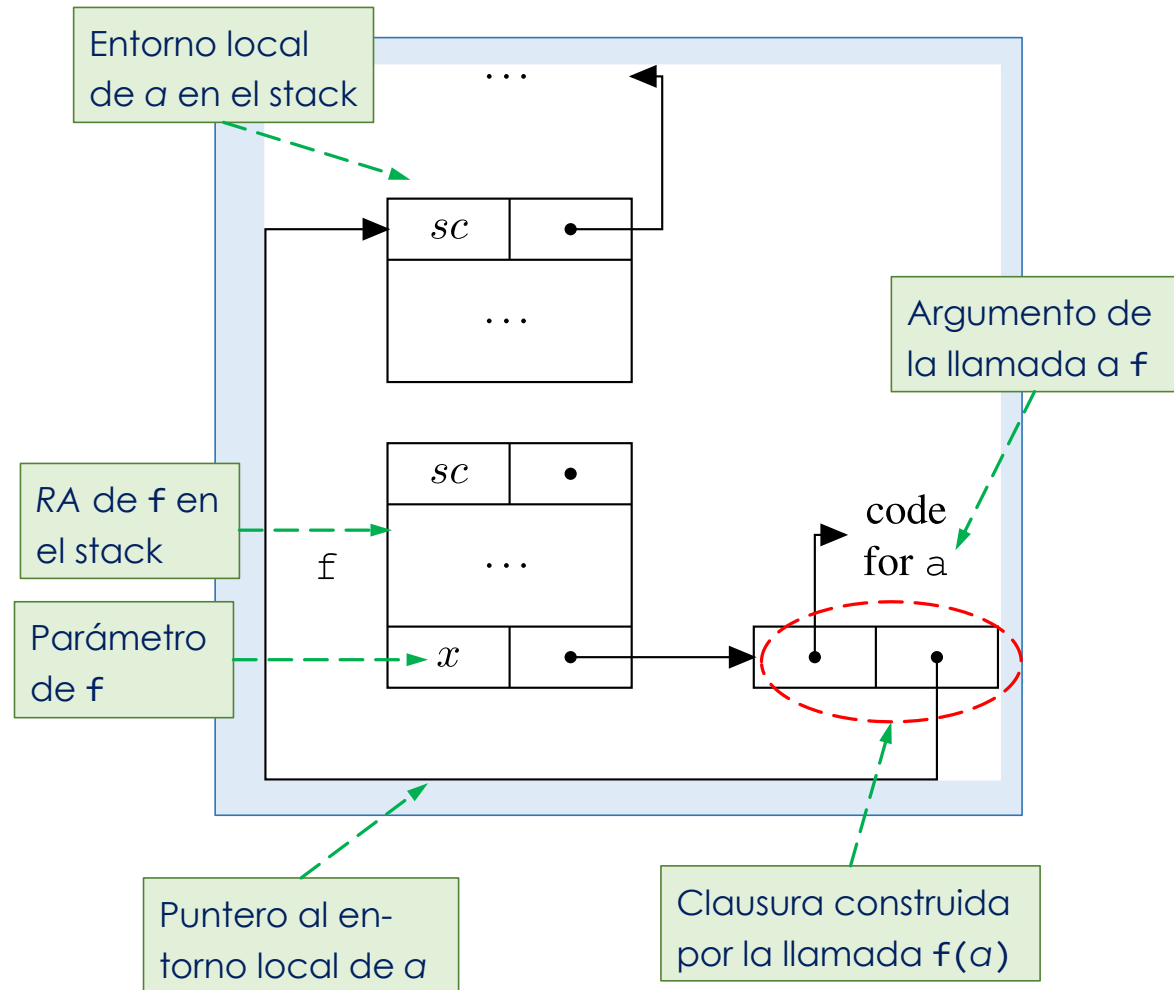
89

El que llama debe pasar no sólo la expresión textual correspondiente al argumento

... sino también el entorno en el cual la expresión debe ser evaluada

→ el par —o la *clausura*— (*expresión*, *entorno*), en que *entorno* incluye todas las variables libres en *expresión*

⇒ Caro



Llamada **por nombre** *no* es
llamada **por valor-resultado**

```
void fiefoo (valueresult/name int x, valueresult/name int y) {  
    x = x+1;  
    y = 1;  
}  
...  
int i = 1;  
int[] A = new int[5];  
A[1]=4;  
fiefoo(i,A[i]);
```

Parámetros pasados por valor-resultado:

`fiefoo(i,A[i])` termina con `A[1]` con valor 1, `i` con valor 2, el resto de `A` no ha sido tocado

En este ej., el mismo comportamiento que por referencia

Parámetros pasados por nombre:

`fiefoo(i,A[i])` termina con `A[1]` con valor 4, `i` con valor 2, y `A[2]` con valor 1

Funciones de orden superior

Cuando acepta como parámetro o retorna como valor una función

Menos común
Fundamental en lenguajes
de programación funcional

Funciones de orden superior

Cuando acepta como parámetro
o retorna como valor una función

Más común

Parámetros funcionales

Declaración de la función g
... con un único parámetro formal h
... que a su vez es una función

```
{int x = 1;
int f(int y){
    return x+y;
}
void g (int h(int b)){
    int x = 2;
    return h(3) + x;
}
...
{int x = 4;
  int z = g(f);
}
```

h es especificada como una
función que retorna un int
... y recibe como parámetro
formal un int

Parámetros funcionales

Declaración de la función g
... con un único parámetro formal h
... que a su vez es una función

f es llamada
a través de h

```
{int x = 1;  
int f(int y){  
    return x+y;  
}  
void g (int h(int b)){  
    int x = 2;  
    return h(3) + x;  
}  
...  
{int x = 4;  
  int z = g(f);  
}
```

h es especificada como una
función que retorna un int
... y recibe como parámetro
formal un int

f pasa como argumento a g

Parámetros funcionales

El nombre `x` está definido varias veces

→ es necesario establecer cuál es el entorno (no local) en el cual `f` será evaluada

```
{int x = 1; ←  
int f(int y){  
    return x+y;  
}  
void g (int h(int b)){  
    int x = 2; ←  
    return h(3) + x;  
}  
...  
{int x = 4; ←  
  int z = g(f);  
}
```


Parámetros funcionales

El nombre `x` está definido varias veces

→ es necesario establecer cuál es el entorno (no local) en el cual `f` será evaluada

```
{int x = 1; ←  
int f(int y){ ←  
    return x+y;  
}  
void g (int h(int b)){  
    int x = 2; ←  
    return h(3) + x;  
}  
...  
{int x = 4; ←  
  int z = g(f);  
}  
}
```

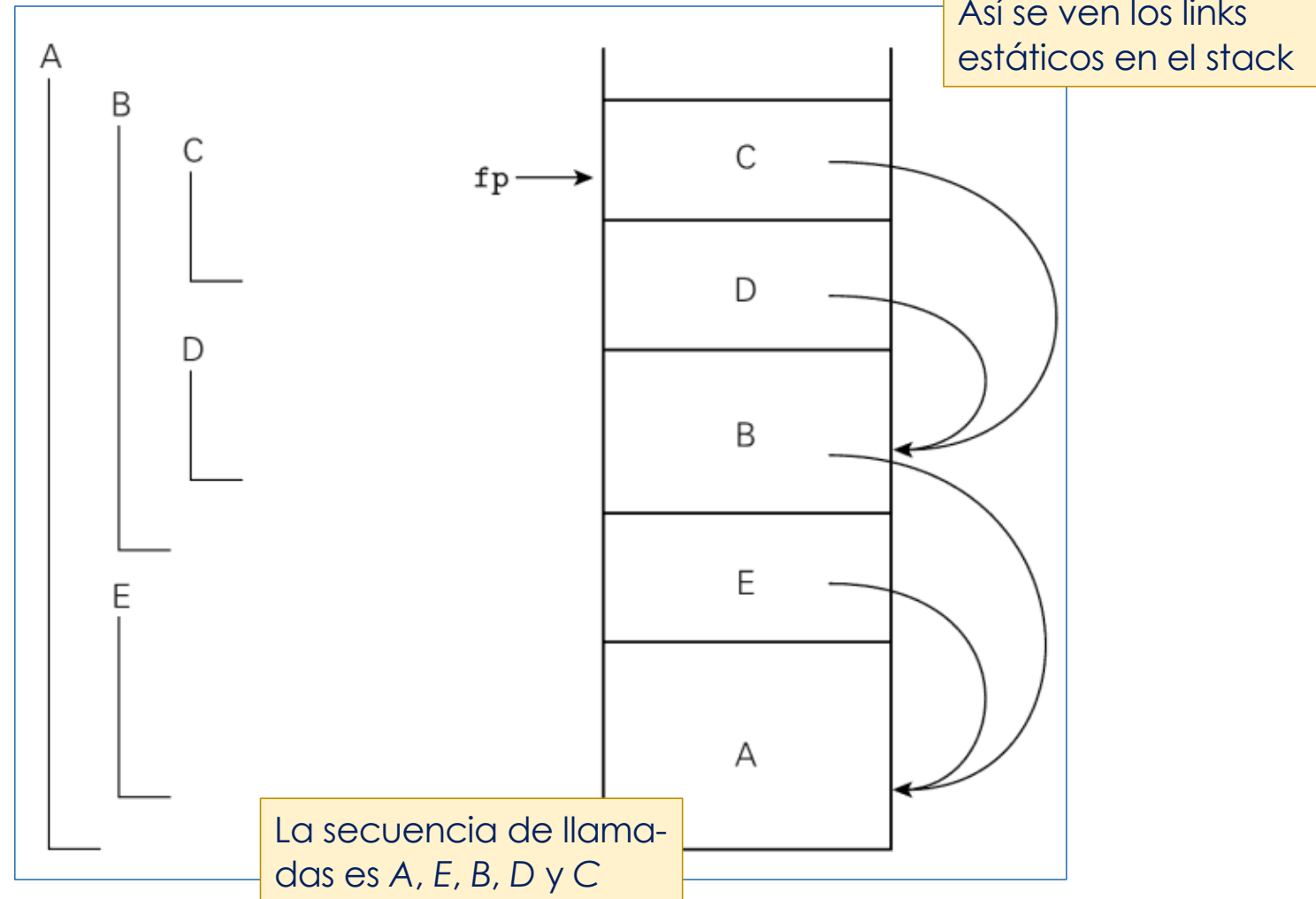
El entorno activo al momento de crear el vínculo entre `h` y `f`:
→ *asociación profunda*

El entorno activo cuando ocurre la llamada a `f` usando `h`:
→ *asociación superficial*

...

Anidación léxica de las subrutinas A, B, C, D y E:
A es léx.circ. a B y a E
B es léx.circ. a C y a D

Alcance y ámbito estáticos



Alcance y ámbito dinámicos

La vinculación vigente para un nombre dado es la que se encuentra más recientemente durante la ejecución y que aún no ha sido destruida por retorno de su alcance

Bajo alcance estático, imprime un 1

Bajo **alcance dinámico**, depende del valor leído en la línea 8:

- si es positivo, imprime un 2
- de lo contrario, un 1

```
1.  n : integer  ¿ Es ésta ?  -- global declaration
2.  procedure first()
3.      n := 1    ¿ Cuál n es esta n ?
4.  procedure second()
5.      n : integer  ¿ ... o es ésta ?  laration
6.      first()
7.  n := 2
8.  if read_integer() > 0
9.      second()
10. else
11.     first()
12. write_integer(n)
```

Alcance y ámbito dinámicos

La asociación para un nombre X , en un punto P del programa, es la asociación más reciente (en sentido temporal) creada para X que aún está activa cuando el flujo del control llega a P

¿ Es ésta ?
A. estático

¿Cuál n es esta n ?

¿ ... o es ésta ?
A. dinámico

```
1.  n : integer          -- global declaration
2.  procedure first()
3.      n := 1
4.  procedure second()
5.      n : integer      -- local declaration
6.      first()
7.  n := 2
8.  if read_integer() > 0
9.      second()
10. else
11.     first()
12. write_integer(n)
```

Bajo **alcance dinámico**, depende del valor leído en la línea 8:

- si es positivo, imprime un 2
- de lo contrario, un 1

Bajo alcance estático, imprime un 1

Parámetros funcionales

```
{int x = 1; ←  
int f(int y){ ←  
    return x+y;  
}  
void g (int h(int b)){ ←  
    int x = 2; ←  
    return h(3) + x;  
}  
...  
{int x = 4; ←  
  int z = g(f);  
}
```

El nombre `x` está definido varias veces

→ es necesario establecer cuál es el entorno (no local) en el cual `f` será evaluada

El entorno activo al momento de crear el vínculo entre `h` y `f`:

→ *asociación profunda*

El entorno activo cuando ocurre la llamada a `f` usando `h`:

→ *asociación superficial*

¿Similar a diferencia entre alcance estático y dinámico?

La política de alcance es independiente de la de asociación:

- los lenguajes con alcance estático usan asociación profunda
- los lenguajes con alcance dinámico usan una u otra

Parámetros funcionales

101

```
{int x = 1;
int f(int y){
    return x+y;
}
void g (int h(int b)){
    int x = 2;
    return h(3) + x;
}
...
{int x = 4;
  int z = g(f);
}
```

Ámbito estático y asociación profunda:

- h(3) retorna 4 (y g, 6)
- la x en f es la del bloque más externo

Ámbito dinámico y asociación superficial:

- h(3) retorna 5 (y g, 7)
- la x en f es la x local a g

Ámbito dinámico y asociación profunda:

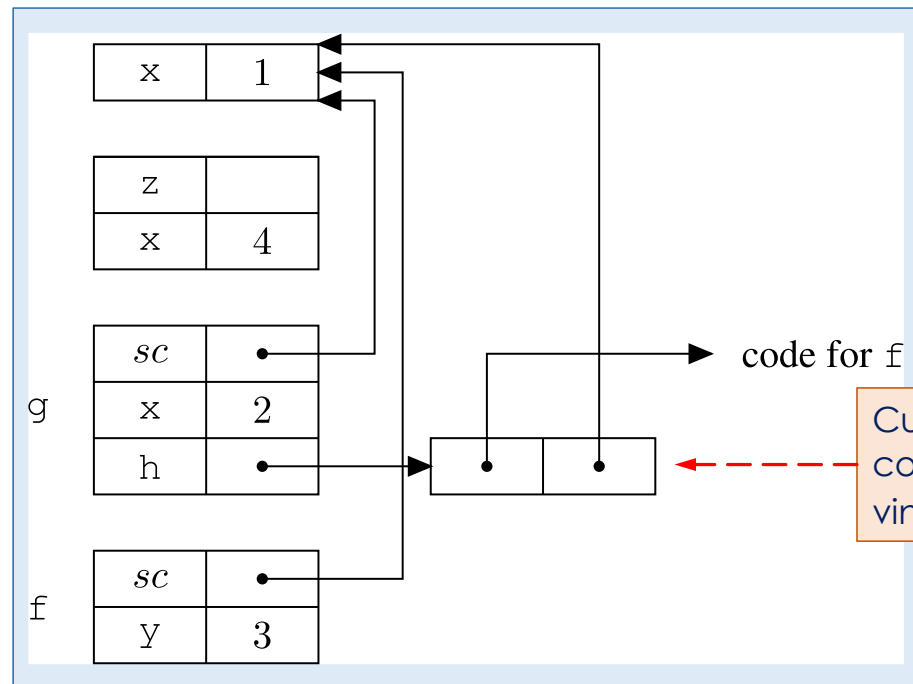
- h(3) retorna 7 (y g, 9)
- la x en f es la x local al bloque en que ocurre g(f)

```

{int x = 1;
int f(int y){
    return x+y;
}
void g (int h(int b)){
    int x = 2;
    return h(3) + x;
}
...
{int x = 4;
  int z = g(f);
}

```

Parámetros funcionales



code for `f`

Cuando `g` es llamada con argumento `f`, se vincula una clausura a `h`

Stack al inicio de la ejecución de `f`, debido a la llamada a `h` en `g`

Funciones de orden superior

Cuando aceptan como parámetro o retornan como valor una función

Menos común

Fundamental en lenguajes
de programación funcional

Funciones de orden superior

Cuando aceptan como parámetro
o retornan como valor una función

Más común

Parámetros de tipo función

Declaración de la función g
... con un único parámetro formal h
... que a su vez es una función

```
{int x = 1;  
int f(int y){  
    return x+y;  
}  
void g (int h(int b)){  
    int x = 2;  
    return h(3) + x;  
}  
...  
{int x = 4;  
  int z = g(f);  
}
```

h es especificada como una
función que retorna un int
... y recibe como parámetro
formal un int

Parámetros de tipo función

94

Declaración de la función g
... con un único parámetro formal h
... que a su vez es una función

f es llamada
a través de h

```
{int x = 1;  
int f(int y){  
    return x+y;  
}  
void g (int h(int b)){  
    int x = 2;  
    return h(3) + x;  
}  
...  
{int x = 4;  
  int z = g(f);  
}
```

h es especificada como una
función que retorna un int
... y recibe como parámetro
formal un int

f pasa como argumento a g

Parámetros de tipo función

95

El nombre `x` está definido varias veces

→ es necesario establecer cuál es el entorno (no local) en el cual `f` será evaluada

```
{int x = 1; ←  
int f(int y){  
    return x+y;  
}  
void g (int h(int b)){  
    int x = 2; ←  
    return h(3) + x;  
}  
...  
{int x = 4; ←  
  int z = g(f);  
}
```

Parámetros de tipo función

El nombre `x` está definido varias veces

→ es necesario establecer cuál es el entorno (no local) en el cual `f` será evaluada:

Hay dos posibilidades:

- Entorno activo *al crear el vínculo entre `h` y `f`*:

→ **asociación profunda**

```
{int x = 1;
int f(int y){
    return x+y;
}
void g (int h(int b)){
    int x = 2;
    return h(3) + x;
}
...
{int x = 4;
  int z = g(f);
}
```

Aquí se crea el vínculo entre `h` y `f`

Parámetros de tipo función

97

El nombre `x` está definido varias veces

→ es necesario establecer cuál es el entorno (no local) en el cual `f` será evaluada:

Hay dos posibilidades:

- Entorno activo *al crear el vínculo entre `h` y `f`*:

→ **asociación profunda**

- Entorno activo *cuando se llama a `f` usando `h`*:

→ **asociación superficial**

...

```
{int x = 1;
int f(int y){
    return x+y;
}
void g (int h(int b)){
    int x = 2;
    return h(3) + x;
}
...
{int x = 4;
  int z = g(f);
}
```

Aquí se llama a `f` usando `h`

Aquí se crea el vínculo entre `h` y `f`

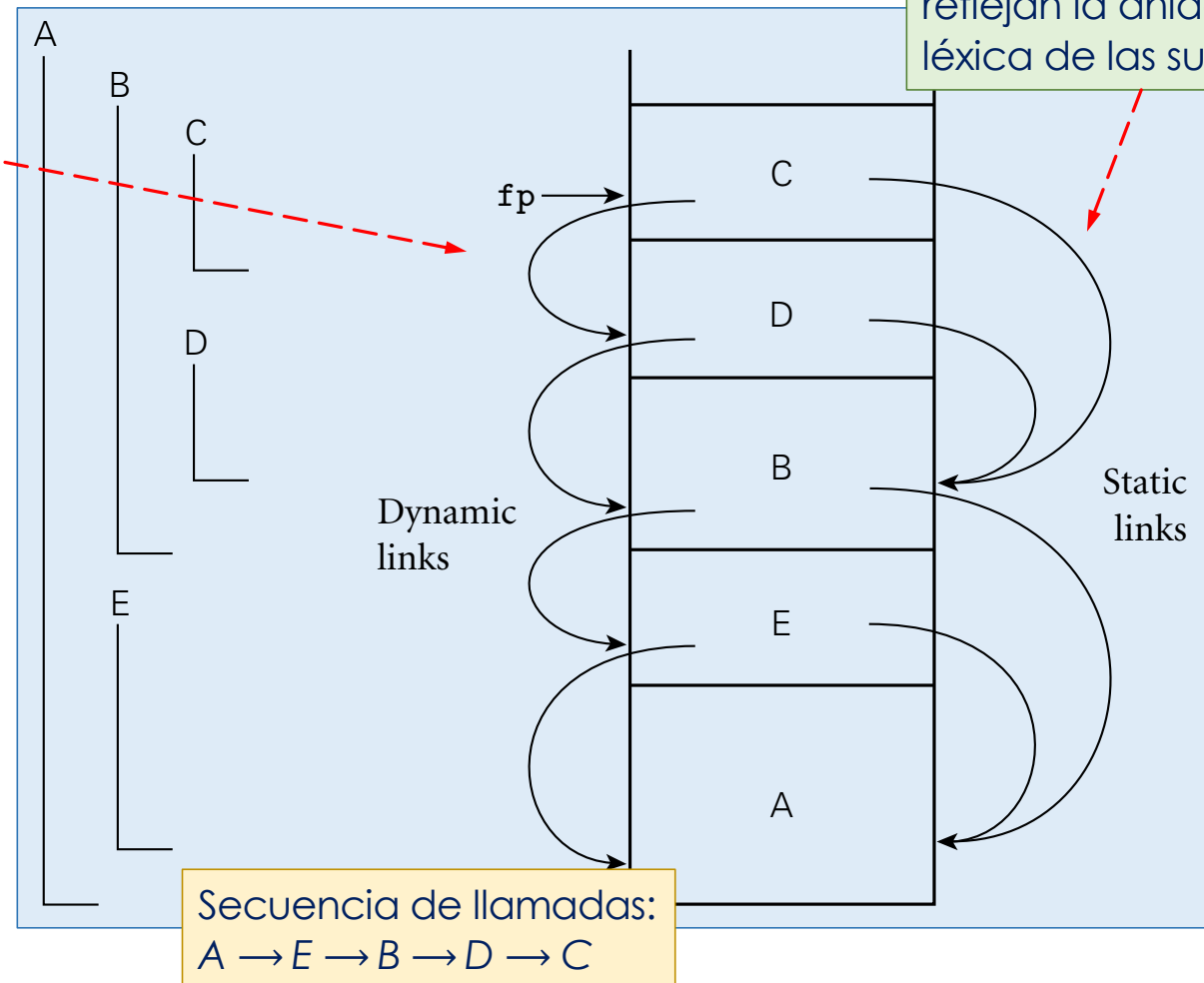
Recordemos: Alcance y ámbito estáticos

Así se ven los links dinámicos en el stack: reflejan la secuencia de llamadas

Anidación léxica de las subrutinas A, B, C, D y E:
A es léx.circ*. a B y a E
B es léx.circ. a C y a D

*léxicamente circundante

Así se ven los links estáticos en el stack: reflejan la anidación léxica de las subrutinas



(también hay) **Alcance y ámbito dinámicos**

La asociación para un nombre X , en un punto P del programa, es la asociación más reciente (en sentido temporal) creada para X que aún está activa cuando el flujo del control llega a P

¿ Es ésta ?
A. estático

¿Cuál n es esta n ?

¿ ... o es ésta ?
A. dinámico

```

1. ➔ n : integer           -- global declaration
2.  procedure first()
3. ➔ ➔ n := 1
4.  procedure second()
5. ➔ ➔ n : integer         -- local declaration
6. ➔ ➔ first()
7. ➔ ➔ n := 2
8. ➔ ➔ if read_integer() > 0
9. ➔ ➔     second()
10. ➔ ➔ else
11. ➔ ➔     first()
12. ➔ ➔ write_integer(n)
  
```

Usado principalmente para simplificar el manejo del entorno en tiempo de ejecución, i.e., links estáticos además de links dinámicos:

- Lisp, APL, SNOBOL, Perl

Bajo **alcance dinámico**, depende del valor leído en la línea 8:

- si es positivo, imprime un 2
- de lo contrario, un 1

Bajo alcance estático, imprime un 1

Parámetros de tipo función

10

El nombre `x` está definido varias veces

→ es necesario establecer cuál es el entorno (no local) en el cual `f` será evaluada:

Hay dos posibilidades:

- Entorno activo *al crear el vínculo entre `h` y `f`*:

→ **asociación profunda**

- Entorno activo *cuando se llama a `f` usando `h`*:

→ **asociación superficial**

```
{int x = 1;
int f(int y){
    return x+y;
}
void g (int h(int b)){
    int x = 2;
    return h(3) + x;
}
...
{int x = 4;
  int z = g(f);
}
```

Aquí se llama a `f` usando `h`

Aquí se crea el vínculo entre `h` y `f`

¿Similar a la diferencia entre alcance estático y dinámico?

La política de alcance es independiente de la de asociación:

- los lenguajes con alcance estático usan asociación profunda
- los lenguajes con alcance dinámico usan una u otra

Asociación superficial se ve como contradictoria

Parámetros de tipo función

101

```
{int x = 1;
int f(int y){
    return x+y;
}
void g (int h(int b)){
    int x = 2;
    return h(3) + x;
}
...
{int x = 4;
  int z = g(f);
}
```

Aquí se crea el
vínculo entre h y f

Bajo ámbito estático, éste es el entorno activo al crear el vínculo entre h y f

→ asociación profunda:

- h(3) retorna 4 (y g, 6)
- la x en f es la del bloque más externo

Parámetros de tipo función

102

```
{int x = 1;
int f(int y){
    return x+y;
}
void g (int h(int b)){
    int x = 2;
    return h(3) + x;
}
...
{int x = 4;
  int z = g(f);
}
```

Aquí se crea el
vínculo entre h y f

Bajo ámbito dinámico, éste es el entorno activo al crear el vínculo entre h y f

→ asociación profunda:

- h(3) retorna 7 (y g, 9)
- la x en f es la x local al bloque en que ocurre g(f)

Parámetros de tipo función

103

```
{int x = 1;
int f(int y){
    return x+y;
}
void g (int h(int b)){
    int x = 2;
    return h(3) + x;
}
...
{int x = 4;
  int z = g(f);
}
```

Aquí se llama
a f usando h

Bajo ámbito dinámico, éste es el entorno activo cuando se llama a f usando h

→ asociación superficial:

- h(3) retorna 5 (y g, 7)
- la x en f es la x local a g

Parámetros de tipo función

104

```
{int x = 1;
int f(int y){
    return x+y;
}
void g (int h(int b)){
    int x = 2;
    return h(3) + x;
}
...
{int x = 4;
  int z = g(f);
}
```

Ámbito estático y asociación profunda:

- h(3) retorna 4 (y g, 6)
- la **x** en f es la del bloque más externo

Ámbito dinámico y asociación superficial:

- h(3) retorna 5 (y g, 7)
- la **x** en f es la x local a g

Siempre
vale 2

Ámbito dinámico y asociación profunda:

- h(3) retorna 7 (y g, 9)
- la **x** en f es la x local al bloque en que ocurre g(f)

Implementación de asociación profunda (en ámbito estático)

```
{int x = 1;
int f(int y){
    return x+y;
}
void g (int h(int b)){
    int x = 2;
    return h(3) + x;
}
...
{int x = 4;
  int z = g(f);
}
```

Diapos. 66 a 68 de “nombres y alcances”

→ cuando *f* es llamada directamente, el compilador calcula un entero *k* : nivel de anidación de la definición de *f* c/r al bloque en que ocurre la llamada

Cuando *f* es llamada indirectamente a través de un parámetro *h*

→ no hay cómo asociar estáticamente un *k* con la llamada —no se sabe cuál va a ser el argumento:

- la información es determinada en el momento de la creación de la asociación entre el parámetro y el argumento —en el ej., cuando ocurre la llamada *g(f)*

Implementación de asociación profunda (en ámbito estático)

```
{int x = 1;
int f(int y){
    return x+y;
}
void g (int h(int b)){
    int x = 2;
    return h(3) + x;
}
...
{int x = 4;
  int z = g(f);
}
```

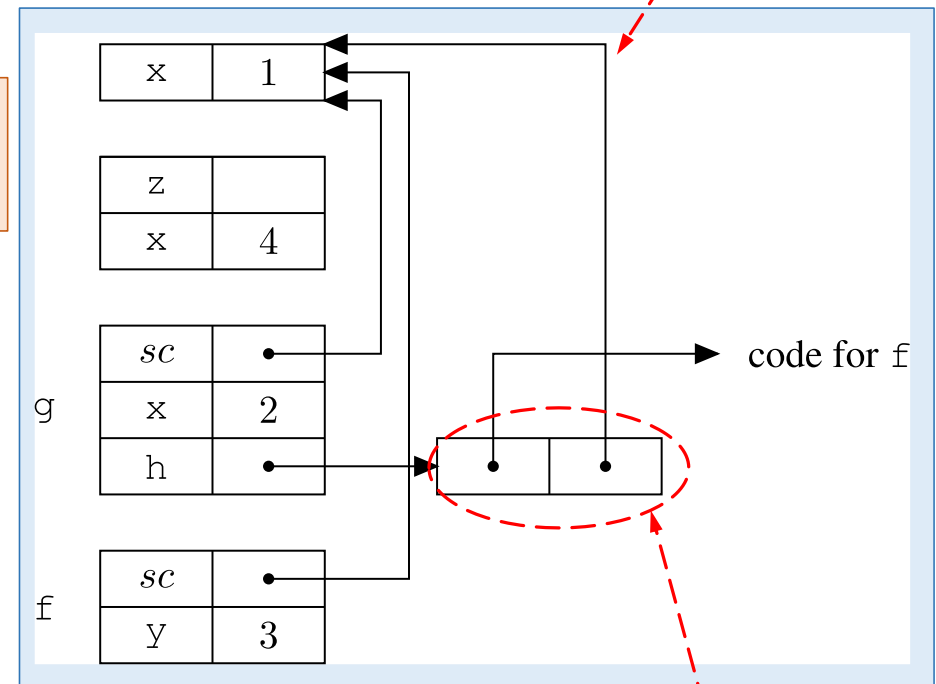
f está declarada a distancia
(de anidación) 1 de donde
aparece como argumento

Cuando f es llamada indirectamente a través de un parámetro h,

→ no hay cómo asociar estáticamente un k con la llamada —no sabemos cuál va a ser el argumento:

- la información es determinada en el momento de la creación de la asociación entre el parámetro y el argumento —en el ej., cuando ocurre la llamada g(f)

Puntero al RA del bloque
en que f está declarada



Stack al inicio de la
ejecución de f (debido
a la llamada a h en g)

Cuando g es llamada
con argumento f, se
vincula una **clausura** a h

Implementación de asociación profunda (en ámbito estático):

- otro ej., en que haría diferencia si la asociación fuera superficial

```
void foo (int f(), int x){
    int fie(){
        return x;
    }
    if (x==0) return f();
    else return foo(fie,0);
}

int g(){
    return 1;
}

print(foo(g,1))
```

Las reglas de ámbito (estático/ dinámico) no permiten decidir:

- la diferencia la hace el tipo de asociación

... en la fig., asociación profunda

¿Cuál debe ser el x de return x?

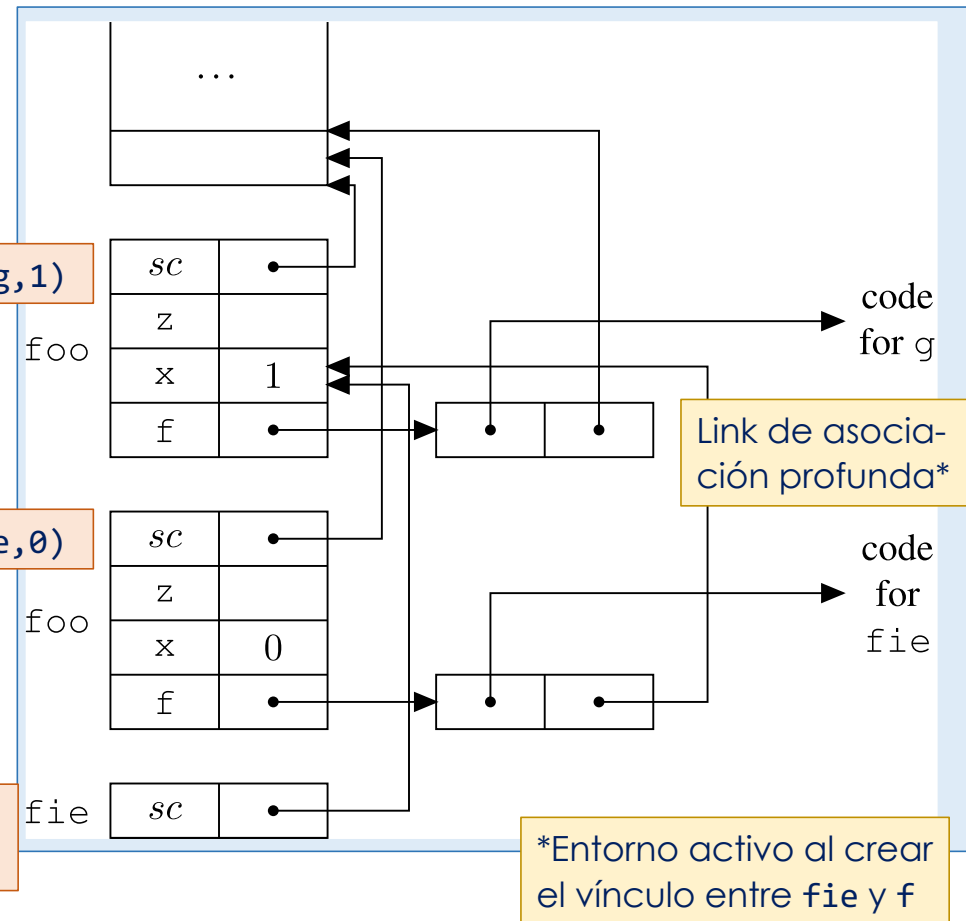
RA para fie(), llamado a través de f()

RA para foo(g,1)

x=1

RA para foo(fie,0)

x=0



Cómo se define el entorno

Reglas de visibilidad, garantizadas normalmente por la estructura de bloques

Excepciones a las reglas de visibilidad (e.g., redefinición de nombres)

Reglas de alcance (o ámbito)

Reglas para el método de paso de parámetros

Política de asociación (profunda o superficial)

Funciones como resultados

Permite la creación dinámica de funciones en tiempo de ejecución (*runtime*)

Una función devuelta como resultado no puede ser representada en tiempo de ejecución sólo por su código:

- hay que incluir el entorno en el cual va a ser evaluada
- (nuevamente) una *clausura*

Funciones como resultados

Una función devuelta como resultado no puede ser representada en tiempo de ejecución sólo por su código:

- hay que incluir el entorno en el cual va a ser evaluada
- una *clausura*

```
{int x = 1;
void->int F () {
    int g () {
        return x+1;
    }
    return g;
}
void->int gg = F();
int z = gg();
}
```

Funciones como resultados

Una función devuelta como resultado no puede ser representada en tiempo de ejecución sólo por su código:

- hay que incluir el entorno en el cual va a ser evaluada
- una *clausura*

Tipo de la función retornada por F: No recibe argumentos y retorna un int

Retorna la *función*, no su aplicación

Declaración del nombre *gg*, que es asociado dinámicamente con el resultado de la evaluación de F

```
{int x = 1;  
void->int F () {  
    int g () {  
        return x+1;  
    }  
    return g;  
}  
void->int gg = F();  
int z = gg();  
}
```

Funciones como resultados

Una función devuelta como resultado no puede ser representada en tiempo de ejecución sólo por su código:

- hay que incluir el entorno en el cual va a ser evaluada
- una *clausura*

Tipo de la función retornada por F: No recibe argumentos y retorna un int

Retorna la *función*, no su aplicación

Declaración del nombre `gg`, que es asociado dinámicamente con el resultado de la evaluación de F

```
{int x = 1;  
void->int F () {  
    int g () {  
        return x+1;  
    }  
    return g;  
}  
void->int gg = F();  
int z = gg();  
}
```

¿ Qué retorna `gg()` ?

Funciones como resultados

Una función devuelta como resultado no puede ser representada en tiempo de ejecución sólo por su código:

- hay que incluir el entorno en el cual va a ser evaluada
- una *clausura*

Tipo de la función retornada por F: No recibe argumentos y retorna un int

Retorna la *función*, no su aplicación

Declaración del nombre `gg`, que es asociado dinámicamente con el resultado de la evaluación de F

```
{int x = 1;  
void->int F () {  
    int g () {  
        return x+1;  
    }  
    return g;  
}  
void->int gg = F();  
int z = gg();  
}
```

Usando la regla de alcance estático:
x está determinado por la estructura del programa

... no por la posición de la llamada a `gg`
... que podría aparecer en un entorno en que hay otra definición del nombre `x`

Funciones como resultados

Una función devuelta como resultado no puede ser representada en tiempo de ejecución sólo por su código:

- hay que incluir el entorno en el cual va a ser evaluada
- una *clausura*

Tipo de la función retornada por F: No recibe argumentos y retorna un int

Retorna la *función*, no su aplicación

Declaración del nombre `gg`, que es asociado dinámicamente con el resultado de la evaluación de F

```
{int x = 1;  
void->int F () {  
    int g () {  
        return x+1;  
    }  
    return g;  
}  
void->int gg = F();  
int z = gg();  
}
```

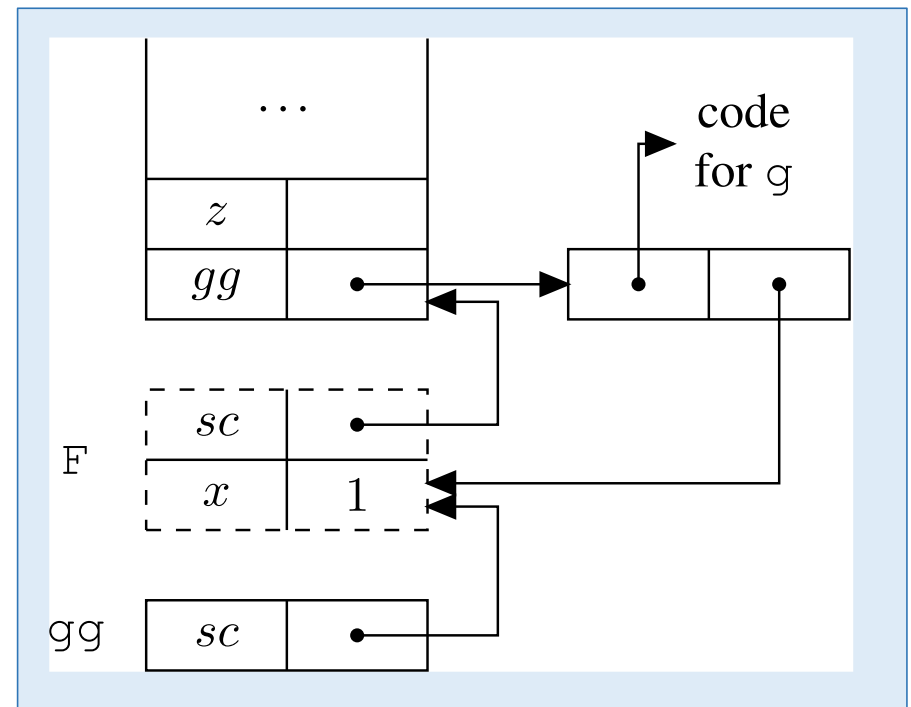
Cuando una función retorna otra como resultado
... este resultado es, en realidad, una clausura

```

void->int F () {
    int x = 1;
    int g () {
        return x+1;
    }
    return g;
}
void->int gg = F();
int z = gg();

```

Funciones como resultados: otro ej.



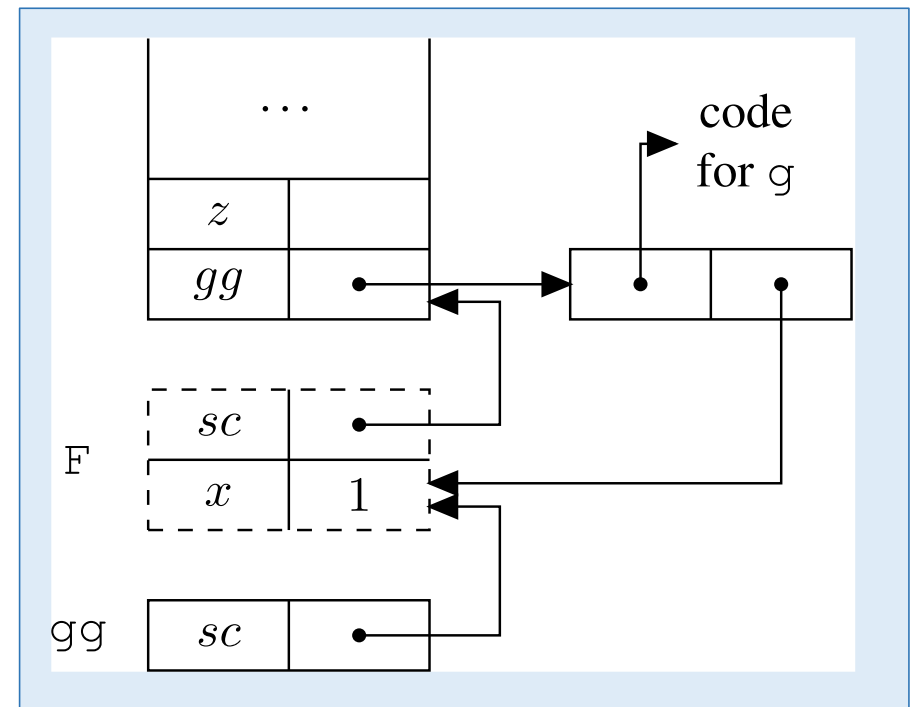
Funciones como resultados: otro ej.

```

void->int F () {
    int x = 1;
    int g () {
        return x+1;
    }
    return g;
}

void->int gg = F();
int z = gg();

```



Ubicar (*allocate*) todos los RAs en el heap y dejar que el recolector de basura los “saque” (*deallocate*) cuando se da cuenta de que ya no hay más referencias a los nombres que contienen